

NATIONAL UNIVERSITY OF SINGAPORE

SCHOOL OF COMPUTING
MIDTERM ASSESSMENT FOR
Semester 1 AY2024/25

CS1101S Programming Methodology

Time Allowed: 1 hour 30 minutes

INSTRUCTIONS TO CANDIDATES

1. This question paper contains 23 questions in 7 sections, and comprises 17 printed pages, including this page.
2. You are also provided with an answer sheet on which to write your answers.
3. Use a pen or pencil to write your Student Number in the designated space on the front page of the answer sheet, and shade the corresponding circle completely in the grid for each digit or letter. **DO NOT WRITE YOUR NAME.**
4. You must submit only the answer sheet and no other documents. Do not tear off any pages from the answer sheet.
5. All questions must be answered in the space provided in the answer sheet; no extra sheets will be accepted as answers.
6. Write legibly with a pen or pencil; do not use red ink. Untidiness will be penalized.
7. For each multiple-choice question, shade the circle corresponding to the one correct answer completely. The number of marks awarded is shown next to each question. There is no penalty for an incorrect answer.
8. The full score of this assessment is **100 marks**.
9. This is a **closed-book** assessment, but you are allowed to bring one double-sided A4 / letter-sized sheet of handwritten or printed notes.
10. Where programs are required, write them in the Python 3 language. A reference of some of the pre-declared functions is given in the appendix of the question paper.
11. Unless otherwise stated, all programs use the Python 3 language.
12. In any question, unless it is specifically allowed, your answer must not use functions given in, or written by you for, other questions.

Section A: Linked-List and Box Notations [8 marks]

1. [4 marks] What is the result of the following program in box notation?

```
lst = llist(llist(1), 2)
pair(tail(lst) if is_llist(head(lst)) else None,
     head(lst) if is_pair(tail(lst)) else 2)
```

- (a.) [None, 2]
 - (b.) [2, 1]
 - (c.) [2, [1, None]]
 - (d.) [[2, None], 2]
 - (e.) [[2, None], 1]
 - (f.) [[2, None], [1, None]] ✓**
 - (g.) [[1, None], [2, None]]
 - (h.) The program leads to an error.
2. [4 marks] What is the result of the following program in linked-list notation?

```
def skipper(xs):
    return (xs if is_none(xs) or is_none(tail(xs))
           else pair(skipper(tail(xs)), skipper(tail(tail(xs)))))
```

```
skipper(llist(1, 2, 3, 4))
```

- (a.) llist(llist(4))
- (b.) llist(llist(llist(llist(4))), 4), llist(4) ✓**
- (c.) llist(llist(llist(None)), 4)
- (d.) llist(llist(llist(4)), 4)
- (e.) llist(llist(llist(None)), None)
- (f.) llist(llist(llist(llist(4),4), llist(4),4), llist(llist(4),4), llist(4),4)
- (g.) llist(4)
- (h.) The program leads to an error.

Section C: Orders of Growth [15 marks]

For each question in this section, choose the order of growth that best characterizes the running time of `fun`. Assume that N is a non-negative integer. In Questions 5 through 7, `xs` is a linked list of length N . Assume that `append` has the following implementation:

```
def append(xs, ys):  
    return (ys if is_none(xs)  
           else pair(head(xs), append(tail(xs), ys)))
```

5. [3 marks]

```
def fun(xs):  
    return xs if is_none(xs) else pair(fun(tail(xs)), tail(xs))
```

- (a.) $\Theta(1)$
- (b.) $\Theta(\log N)$
- (c.) $\Theta(N)$ ✓
- (d.) $\Theta(N \log N)$
- (e.) $\Theta(N^2)$
- (f.) $\Theta(N^2 \log N)$
- (g.) $\Theta(N^3)$
- (h.) $\Theta(k^N)$

6. [3 marks]

```
def fun(xs):  
    return (xs if is_none(xs)  
           else append(l1list(head(xs)), fun(tail(xs))))
```

- (a.) $\Theta(1)$
- (b.) $\Theta(\log N)$
- (c.) $\Theta(N)$ ✓
- (d.) $\Theta(N \log N)$
- (e.) $\Theta(N^2)$
- (f.) $\Theta(N^2 \log N)$
- (g.) $\Theta(N^3)$
- (h.) $\Theta(k^N)$

7. [3 marks]

```
def fun(xs):  
    return (xs if is_none(xs)  
            else append(fun(tail(xs)), llist(head(xs))))
```

- (a.) $\Theta(1)$
- (b.) $\Theta(\log N)$
- (c.) $\Theta(N)$
- (d.) $\Theta(N \log N)$
- (e.) $\Theta(N^2)$ ✓**
- (f.) $\Theta(N^2 \log N)$
- (g.) $\Theta(N^3)$
- (h.) $\Theta(k^N)$

8. [3 marks]

```
def fun(N):  
    return (1 if N <= 1  
            else N * (fun(math_floor(N / 2)) + fun(math_floor(N /  
2))))
```

- (a.) $\Theta(1)$
- (b.) $\Theta(\log N)$
- (c.) $\Theta(N)$ ✓**
- (d.) $\Theta(N \log N)$
- (e.) $\Theta(N^2)$
- (f.) $\Theta(N^2 \log N)$
- (g.) $\Theta(N^3)$
- (h.) $\Theta(k^N)$

9. [3 marks]

```
def fun(N):  
    return (1 if N <= 1  
            else N * fun(N / 4) if N % 4 == 0  
            else N + fun(N - 8))
```

(a.) $\Theta(\log N)$

(b.) $\Theta(N)$

(c.) $\Theta(N \log N)$

(d.) $\Theta(N^2)$

(e.) $\Omega(N)$

(f.) $\Omega(N \log N)$

(g.) $O(\log N)$

(h.) $O(N)$ ✓

Section D: Sorting [12 marks]

```
def insert(x, xs, cmp):
    return (llist(x) if is_none(xs)
            else pair(x, xs) if cmp(x, head(xs))
            else pair(head(xs), insert(x, tail(xs), cmp)))

def insert_sort(xs, cmp):
    return (xs if is_none(xs)
            else insert(head(xs), insert_sort(tail(xs), cmp), cmp))
```

10. [4 marks] What is the result of the following application?

```
L = llist(6, 8, 5, 8, 8, 1, 6, 2, 6, 7)
insert_sort(L, lambda x, y: x == y)
```

- (a.) llist(6, 8, 5, 8, 8, 1, 6, 2, 6, 7)
- (b.) llist(7, 6, 2, 6, 1, 8, 8, 5, 8, 6)
- (c.) llist(5, 8, 8, 8, 1, 2, 6, 6, 6, 7)
- (d.) llist(7, 6, 6, 6, 2, 1, 8, 8, 8, 5) ✓**
- (e.) llist(6, 6, 6, 8, 8, 8, 5, 1, 2, 7)
- (f.) llist(7, 2, 1, 5, 8, 8, 8, 6, 6, 6)
- (g.) llist(8, 8, 8, 6, 6, 6, 5, 1, 2, 7)
- (h.) llist(7, 2, 1, 5, 6, 6, 6, 8, 8, 8)

11. [4 marks] What is the result of the following application?

```
L = llist(pair(9, 2), pair(5, 7), pair(3, 5), pair(5, 9),
          pair(1, 4), pair(3, 2), pair(5, 8))
insert_sort(L, lambda x, y: head(x) < head(y))
```

- (a.) llist([1,4], [3,2], [3,5], [5,8], [5,9], [5,7], [9,2]) ✓**
- (b.) llist([1,4], [3,5], [3,2], [5,7], [5,9], [5,8], [9,2])
- (c.) llist([1,4], [3,2], [3,5], [5,7], [5,8], [5,9], [9,2])
- (d.) llist([1,4], [3,5], [3,2], [5,9], [5,8], [5,7], [9,2])
- (e.) llist([1,4], [3,2], [3,5], [5,9], [5,8], [5,7], [9,2])
- (f.) llist([1,4], [3,5], [3,2], [5,7], [5,8], [5,9], [9,2])
- (g.) llist([1,4], [3,2], [3,5], [5,9], [5,7], [5,8], [9,2])
- (h.) llist([1,4], [3,5], [3,2], [5,8], [5,7], [5,9], [9,2])

12. [4 marks] What is the result of the following application?

```
def cmp(x, y):  
    return (head(x) > head(y)  
           or (head(x) == head(y) and tail(x) > tail(y)))  
  
L = llist(pair(9, 2), pair(5, 7), pair(3, 5), pair(5, 9),  
          pair(1, 4), pair(3, 2), pair(5, 8))  
insert_sort(L, cmp)
```

- (a.) `llist([9,2], [5,8], [5,9], [5,7], [3,2], [3,5], [1,4])`
- (b.) `llist([9,2], [5,9], [5,8], [5,7], [3,5], [3,2], [1,4])` ✓**
- (c.) `llist([9,2], [5,7], [5,9], [5,8], [3,5], [3,2], [1,4])`
- (d.) `llist([9,2], [5,7], [5,8], [5,9], [3,2], [3,5], [1,4])`
- (e.) `llist([1,4], [3,5], [3,2], [5,9], [5,8], [5,7], [9,2])`
- (f.) `llist([1,4], [3,2], [3,5], [5,7], [5,8], [5,9], [9,2])`
- (g.) `llist([5,9], [5,8], [5,7], [3,5], [1,4], [9,2], [3,2])`
- (h.) `llist([5,9], [5,8], [5,7], [3,5], [1,4], [3,2], [9,2])`

Section E: Active Lists [10 marks]

An active list maps an integer position to `None` or a one-element list. Its first element is at position 0. For example, if `A` represents `l1ist(8, 3, 5)`, then `A(0)`, `A(1)`, and `A(2)` return `l1ist(8)`, `l1ist(3)`, and `l1ist(5)`, respectively; `A(-1)` and `A(3)` return `None`. The pre-declared function `make_active_list(xs)` constructs an active list from `xs`, and `active_length(A)` returns the smallest non-negative integer `k` for which `A(k)` is `None`.

13. [4 marks] The function `active_reverse(A)` returns an active list containing the same elements as `A` in reverse order. Which expression correctly completes it?

For example, if `A` represents `l1ist(8, 3, 5, 7)`, the result represents `l1ist(7, 5, 3, 8)` and returns `None` for positions `-1` and `4`.

```
def active_reverse(A):
    size = active_length(A)
    return None # replace None with an expression
```

- (a.) `A(lambda p: p - size + 1)`
- (b.) `A(lambda p: size - p - 1)`
- (c.) `l1ist(lambda p: A(p - size + 1))`
- (d.) `l1ist(lambda p: A(size - p - 1))`
- (e.) `lambda p: A(p - size + 1)`
- (f.) `lambda p: A(size - p - 1) ✓`
- (g.) `lambda p: l1ist(A(p - size + 1))`
- (h.) `lambda p: l1ist(A(size - p - 1))`

14. [6 marks] Complete `insert_elem(A, pos, elem)`, which returns an active list containing the elements of `A` with `elem` inserted at position `pos`. The resulting active list has length one more than `A`. Assume that `pos` is non-negative and no greater than `active_length(A)`. You must not use `make_active_list`.

For example, inserting 99 at position 1 into an active list representing `l1ist(8, 3, 5)` gives an active list representing `l1ist(8, 99, 3, 5)`.

```
def insert_elem(A, pos, elem):
    return ____[1]____
```

Solution:

```
def insert_elem(A, pos, elem):
    return lambda p: (A(p) if p < pos
                      else l1ist(elem) if p == pos
                      else A(p - 1))
```

Section F: Trees [25 marks]

15. [4 marks] Which of the following data structures are trees of numbers?

```
data1 = llist(0, llist(1, 2), llist(3, None))
data2 = pair(llist(1, 2), pair(3, llist(4, 5)))
data3 = pair(None, pair(pair(1, 2), pair(3, None)))
```

- (a.) data1 only
- (b.) data2 only
- (c.) data3 only
- (d.) data1 and data2 only ✓**
- (e.) data1 and data3 only
- (f.) data2 and data3 only
- (g.) data1, data2, and data3
- (h.) None of them

16. [6 marks] Complete `is_num_tree(data)`, which returns `True` exactly when `data` is a tree of numbers. The empty tree is a tree of numbers; for example, `llist(1, 2, None, llist(3, 4))` is one, but `llist(None, "4")` is not. You must not use higher-order linked-list processing functions. You may find `is_number` and `is_pair` useful.

```
def is_num_tree(data):
    return (___[1]___ if is_none(data)
            else ___[2]___ if is_pair(data)
            else ___[3]___)
```

Solution:

```
def is_num_tree(data):
    return (True if is_none(data)
            else ((is_number(head(data)) or
is_num_tree(head(data)))
                 and is_num_tree(tail(data)))
            if is_pair(data)
            else False)
```

17. [4 marks] What is the value of result after map_tree(llist, tree)?

```
def map_tree(f, tree):  
    return map(lambda sub_tree:  
                (f(sub_tree) if not is_llist(sub_tree)  
                 else map_tree(f, sub_tree)),  
                tree)
```

```
tree = llist(1, 2, llist(3, 4))  
result = map_tree(llist, tree)
```

- (a.) llist(llist(1), llist(2), llist(llist(3), llist(4))) ✓
- (b.) llist(llist(1), llist(2), llist(llist(3, 4)))
- (c.) llist(llist(1), llist(2), llist(3, 4))
- (d.) llist(1, 2, llist(3, 4))
- (e.) llist(llist(1, 2, llist(3, 4)))
- (f.) llist(llist(1, 2, llist(llist(3, 4))))
- (g.) None of the above

18. [6 marks] Complete `count_elem(elem, tree)`, which takes a number `elem` and a tree of numbers and returns the number of times `elem` appears in the tree. You must use `accumulate_tree`. For example:

```
tree = llist(11, llist(11, llist(22, 33), llist(11, llist(22, 44))))
count_elem(77, tree) # returns 0
count_elem(11, tree) # returns 3
count_elem(33, tree) # returns 1
```

```
def accumulate_tree(f, op, initial, tree):
    return reduce(
        lambda x, ys:
            (op(f(x), ys) if not is_llist(x)
             else op(accumulate_tree(f, op, initial, x), ys)),
        initial,
        tree)
```

```
def count_elem(elem, tree):
    return accumulate_tree(
        __[1]__,
        __[2]__,
        __[3]__,
        __[4]__)
```

Solution:

```
def count_elem(elem, tree):
    return accumulate_tree(lambda x: 1 if x == elem else 0,
                           lambda x, y: x + y,
                           0,
                           tree)
```

19. [5 marks] What sequence is printed by the given `accumulate_tree` application?

```
tree = llist(8, llist(5, 4), llist(llist(2), 3))
def print_value(x):
    print(x)
    return x
```

```
accumulate_tree(lambda x: x, lambda x, y: print_value(x), 1, tree)
```

- (a.) 3 2 4 5 8
- (b.) 1 3 2 4 5 8
- (c.) 3 2 2 2 2 4 8 5
- (d.) 3 2 2 4 4 5 5 8
- (e.) 3 2 2 2 4 5 5 8 ✓**
- (f.) 3 2 2 2 4 5 8 8
- (g.) 2 2 2 3 4 5 5 8
- (h.) 2 2 2 3 4 5 8 5

Section G: Functions and More [20 marks]

20. [5 marks] The recursive function `repeat_pattern` applies `pattern` exactly `n` times to `initial`:

```
def repeat_pattern(n, pattern, initial):  
    return (initial if n == 0  
            else pattern(repeat_pattern(n - 1, pattern, initial)))  
  
repeat_pattern(5, lambda x: x * 2, 1) # returns 32
```

Which expression correctly fills the `reduce` callback in the proposed alternative implementation below?

```
def repeat_pattern(n, pattern, initial):  
    xs = build_llist(lambda x: 0, n)  
    return reduce(  
        None, # choose an expression  
        initial,  
        xs)
```

- (a.) `lambda x, y: x(y, pattern)`
 - (b.) `lambda x, y: y(x, pattern)`
 - (c.) `lambda x, y: x(pattern)`
 - (d.) `lambda x, y: y(pattern)`
 - (e.) `lambda x, y: pattern(x)`
 - (f.) `lambda x, y: pattern(y)` ✓**
21. [5 marks] What is the result of the given Python program with `f`, `g`, and `h`?

```
f = lambda x: h(x + 3)  
g = lambda y: f(y + 2)  
h = lambda z: z + 1  
g(f(3))
```

- (a.) 9
- (b.) 10
- (c.) 11
- (d.) 12
- (e.) 13 ✓**
- (f.) 14
- (g.) 15
- (h.) None of the above

22. [4 marks] Which one of the following programs terminates?

- (a.) `(lambda x: x(x))(lambda x: x)(lambda x: x(x))` ✓
- (b.) `(lambda x: x)(lambda x: x(x))(lambda x: x(x))`
- (c.) `(lambda x: x)((lambda x: x(x))(lambda x: x(x)))`
- (d.) `(lambda x: x(x))((lambda x: x)(lambda x: x(x)))`
- (e.) `(lambda x: x(x))(lambda x: x(x))(lambda x: x)`
- (f.) `(lambda x: x(x(x)))(lambda x: x(x))`

23. [6 marks] A number triangle is a non-empty list of rows, starting with a one-element row and growing by one element per row. Complete `rotate(xss)` so that it rotates a number triangle 120 degrees counter-clockwise. You may use only `map`, `head`, `tail`, and `xss` in the missing expressions.

For example:

```
triangle = llist(llist(10),
                 llist(20, 30),
                 llist(40, 50, 60),
                 llist(70, 80, 90, 99))
```

```
rotate(triangle)
# returns llist(llist(99),
#             llist(60, 90),
#             llist(30, 50, 80),
#             llist(10, 20, 40, 70))
```

```
def rotate(xss):
    return (xss if is_none(xss)
            else append(
                rotate(__[1]__),
                llist(__[2]__)))
```

Solution:

```
def rotate(xss):
    return (xss if is_none(xss)
            else append(
                rotate(map(tail, tail(xss))),
                llist(map(head, xss))))
```

APPENDIX

Python §2 Function Reference

Pairs, linked lists, higher-order functions, type predicates, and mathematical helpers

The following functions are pre-declared in Python §2. **None** denotes the empty linked list; linked-list positions start at zero. The linked-list processing functions below use iterative processes, with the stated time and space bounds. Bounds assume constant-time element operations.

Pairs and linked lists

pair(x, y) Constructs a pair with head **x** and tail **y**. Time and space $O(1)$.

is_pair(x) Returns **True** if **x** is a pair, and **False** otherwise. Time and space $O(1)$.

head(x) Returns the first component of pair **x**. Time and space $O(1)$.

tail(x) Returns the second component of pair **x**. Time and space $O(1)$.

is_none(xs) Returns **True** if **xs** is **None**, and **False** otherwise. Time and space $O(1)$.

is_llist(x) Returns **True** if **x** is a linked list as defined in lectures, and **False** otherwise. Time $O(n)$ and space $O(1)$, where **n** is the length of the chain of tail operations that can be applied to **x**.

llist(x1, ..., xn) Constructs a linked list of **n** elements, in the given order. Time and space $O(n)$; the result contains **n** pairs.

length(xs) Returns the length of linked list **xs**. Time $O(n)$ and space $O(1)$, where **n** is its length.

Search and selection

member(x, xs) Returns the first suffix whose head is identical to **x**, or **None** if none exists. Scalar values match by value; pairs and functions match by object identity. Time $O(n)$ and space $O(1)$.

remove(x, xs) Returns a linked list with the first matching element removed, if present. Matching uses scalar value equality and pair/function identity. Time and space $O(n)$.

remove_all(x, xs) Returns a linked list with every matching element removed, using scalar value equality and pair/function identity. Time and space $O(n)$.

filter(pred, xs) Returns the elements of **xs** for which unary predicate **pred** returns **True**, preserving their order. Time and space $O(n)$.

enum_llist(start, end) Enumerates numbers from **start** in steps of 1 until the next number exceeds **end**. The end is inclusive: **enum_llist(2, 5)** returns **llist(2, 3, 4, 5)**. Time and space $O(n)$ in the result length.

llist_ref(xs, n) Returns the element at position **n**, counting from zero. Time $O(n)$ and space $O(1)$.

Building and processing linked lists

map(*f*, *xs*) Returns the result of applying unary function *f* to every element of *xs*, in order. Time and space $O(n)$.

build_llist(*f*, *n*) Constructs *n* elements by applying *f* to the numbers 0 through *n* - 1, in order. Time and space $O(n)$.

for_each(*f*, *xs*) Applies *f* to each element of *xs* in order, then returns **True**. Time $O(n)$ and space $O(1)$.

reverse(*xs*) Returns the elements of *xs* in reverse order. Time and space $O(n)$; the process is iterative, but the result itself occupies $O(n)$ space.

append(*xs*, *ys*) Returns the elements of *xs* followed by those of *ys*. Time and space $O(n)$ in the length of *xs*. A definition supplied in a question overrides this pre-declared implementation.

reduce(*op*, *initial*, *xs*) Folds from right to left. First *op* is applied to the last element and *initial*, then to the preceding element and that result, and so on. For example:

```
reduce(op, initial, llist(1, 2, 3))  
# equals op(1, op(2, op(3, initial)))
```

Time and space $O(n)$, assuming *op* takes constant time. **reduce** here is the pre-declared linked-list right fold.

Types, output, and mathematics

is_boolean(*x*), is_number(*x*), is_string(*x*), is_function(*x*) Return **True** exactly when *x* has the named type, and **False** otherwise. **is_integer**, **is_float**, and **is_complex** test the corresponding numeric types.

print(*objects) Prints the objects followed by a newline and returns **None**. Where a question needs to print and return a value, it defines **print_value** explicitly.

str(object), repr(object) Return informal and debugging string representations, respectively.

llist_to_string(*xs*) Returns a string representing *xs* in box notation.

math_floor(*x*) Returns the largest integer less than or equal to *x*.

math_sqrt(*x*) Returns the square root of *x*.

math_pow(*base*, *exp*) Returns *base* raised to the power *exp*.

min(*x1*, ..., *xn*), max(*x1*, ..., *xn*) Return the smallest and largest supplied values, respectively. Time $O(n)$ and space $O(1)$.

END OF PAPER